# Intelligent and Communicating Systems (ICS)

**LAB N°03 – Arduino Communications (Force Sensor – Interruptions)**

**Full Name: Amel Sadoun – Group 2SQ2**

## A. THEORY

### 1. Force Sensor: Definition, Principle, Category, and Connection

A **Force Sensor** or **Force-Sensitive Resistor (FSR)** is a variable resistor whose resistance decreases when pressure or force is applied to its surface. It belongs to the category of **analog sensors**, producing a voltage proportional to the applied force when connected in a voltage divider with a fixed resistor.

**Principle:** When force is applied, the conductive polymer of the FSR compresses, reducing resistance from several MΩ (no pressure) to a few kΩ (strong pressure). The Arduino reads this voltage via an `analogRead()` function to quantify the applied force.

**Connection:** The FSR is connected as a voltage divider between 3.3V and GND, with the junction point connected to an analog input (e.g., A0). The resulting voltage varies with force.

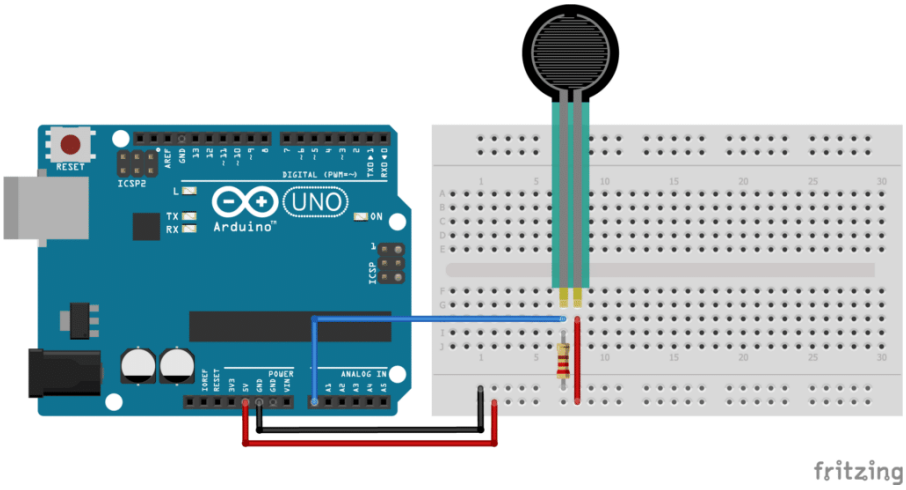


**Figure 1 – Force sensor (FSR) connected to Arduino analog pin**

### 2. Interruption

**2.1 Definition:** An **interrupt** is a signal that temporarily stops the main program to execute a specific function called an **Interrupt Service Routine (ISR)**. It allows the microcontroller to respond immediately to asynchronous events (e.g., button press) without continuously polling the input state.

**2.2 Arduino Interrupt Pins:** On most Arduino boards (including the MKR series), hardware interrupts are supported on specific digital pins such as D0–D3. Interrupts can be configured to trigger on:

- `RISING` – when the signal changes from LOW to HIGH,
- `FALLING` – when the signal changes from HIGH to LOW,
- `CHANGE` – on any transition,
- `LOW` – when the signal stays LOW.

**2.3 Program Structure:** Interrupts use the `attachInterrupt()` function:

```
attachInterrupt(digitalPinToInterrupt(pin), ISR_name, mode);
```

where `ISR_name` is the function executed during the interrupt. Within the ISR, minimal and fast operations are recommended—usually setting a `volatile` flag variable that is later handled in the main loop.

## B. ACTIVITY

### 1. Force Sensor (Analog Input with LED)

**Objective:** Turn ON an LED when the force applied to the FSR exceeds a given threshold, otherwise turn it OFF, and display the readings on the Serial Monitor.

**Circuit:**

- FSR connected between 3.3 V and A0.
- 10 kΩ fixed resistor between A0 and GND (voltage divider).
- LED connected to digital pin 7.

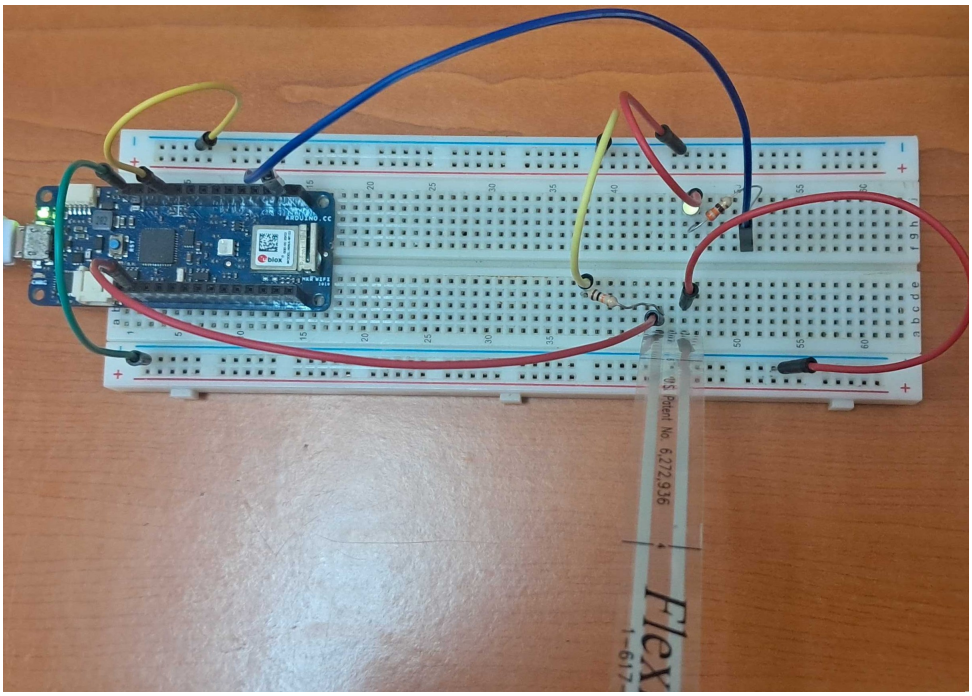

**Figure 2 – FSR circuit with LED indicator**

**Code Example:**

Listing 1: Simple force sensor with LED threshold control

```
const int fsrPin = A0;
const int ledPin = 7;
const int threshold = 20;

void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
```

```
  int fsrValue = analogRead(fsrPin);
  Serial.print("FSR raw: ");
  Serial.println(fsrValue);

  if (fsrValue > threshold)
    digitalWrite(ledPin, HIGH);
  else
    digitalWrite(ledPin, LOW);

  delay(100);
}
```

**Explanation:** The LED lights up when the analog value exceeds the set threshold, representing a strong applied force. The serial monitor displays the real-time sensor values.

### 2. Pushbutton with Internal Pull-Up

**Objective:** Use a pushbutton configured with Arduino's internal pull-up resistor to control a second LED. The LED turns ON briefly when the button is **released**.

**Circuit:**

- Button connected between pin 6 and GND.
- Internal pull-up activated using `INPUT_PULLUP`.
- LED connected to pin 5.

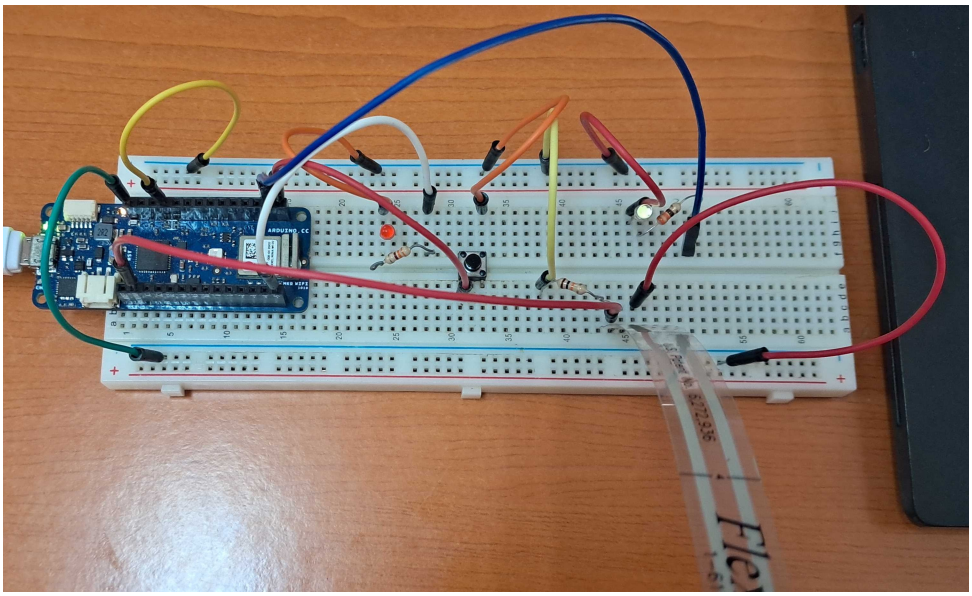

**Figure 3 – Pushbutton with internal pull-up controlling LED**

**Code Example:**

Listing 2: Force sensor and pushbutton using internal pull-up

```
const int fsrPin = A0;
const int ledFSR = 7;
const int buttonPin = 6;
const int ledButton = 5;
const int fsrThreshold = 10;

int fsrValue = 0;
```

```
int buttonState = HIGH;
int lastButtonState = HIGH;

void setup() {
  pinMode(fsrPin, INPUT);
  pinMode(ledFSR, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);
  pinMode(ledButton, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  fsrValue = analogRead(fsrPin);
  Serial.print("FSR Value: ");
  Serial.print(fsrValue);

  if (fsrValue > fsrThreshold)
    digitalWrite(ledFSR, HIGH);
  else
    digitalWrite(ledFSR, LOW);

  buttonState = digitalRead(buttonPin);

  if (lastButtonState == LOW && buttonState == HIGH) {
    Serial.print("  |  Button released!");
    digitalWrite(ledButton, HIGH);
    delay(200);
    digitalWrite(ledButton, LOW);
  }

  lastButtonState = buttonState;
  Serial.println();
  delay(100);
}
```

**Explanation:** This program combines both the FSR and pushbutton control. The button LED reacts on release thanks to the internal pull-up logic (active LOW), simplifying the circuit wiring.

### 3. Interruptions on Pushbutton

**Objective:** Use an interrupt-based system to detect button releases instantly and activate the LED accordingly.

**Circuit:** Same as before, but the button pin must support hardware interrupts (e.g., pin 6 or D2–D3).

**Code Example:**

Listing 3: FSR and pushbutton using interrupt on RISING edge

```
const int fsrPin = A0;
const int ledFSR = 7;
const int buttonPin = 6;
const int ledButton = 5;
const int fsrThreshold = 5;

volatile bool buttonReleased = false;
int fsrValue = 0;

void setup() {
  pinMode(fsrPin, INPUT);
```

```
  pinMode(ledFSR, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);
  pinMode(ledButton, OUTPUT);
  Serial.begin(9600);

  attachInterrupt(digitalPinToInterrupt(buttonPin), onButtonRelease, RISING)
      ;
}

void loop() {
  fsrValue = analogRead(fsrPin);
  Serial.print("FSR Value: ");
  Serial.print(fsrValue);

  if (fsrValue > fsrThreshold)
    digitalWrite(ledFSR, HIGH);
  else
    digitalWrite(ledFSR, LOW);

  if (buttonReleased) {
    buttonReleased = false;
    Serial.print("  |  Button released (interrupt)");
    digitalWrite(ledButton, HIGH);
    delay(200);
    digitalWrite(ledButton, LOW);
  }

  Serial.println();
  delay(100);
}

void onButtonRelease() {
  buttonReleased = true;
}
```

**Explanation:** Here, the program no longer checks the button state continuously. Instead, the interrupt detects the **RISING edge** (button release) and executes the ISR immediately. This method ensures fast and efficient response, even during intensive sensor readings.

### 4. Observations and Saturation Test

Testing different combinations of sensor pressure and button presses shows that the interrupt-based system maintains accurate response even under rapid events, while polling-based code may miss some transitions.

## C. CONCLUSION

In this lab, we integrated analog sensing (FSR) with digital inputs (pushbutton) and learned to handle asynchronous events using hardware interrupts. We observed that:

- FSRs convert applied force into variable resistance measurable via analog input.
- Internal pull-up resistors simplify button circuits by avoiding floating inputs.
- Interrupts offer precise, non-blocking event detection.
- Combining sensor data and interrupts builds efficient and responsive embedded systems.